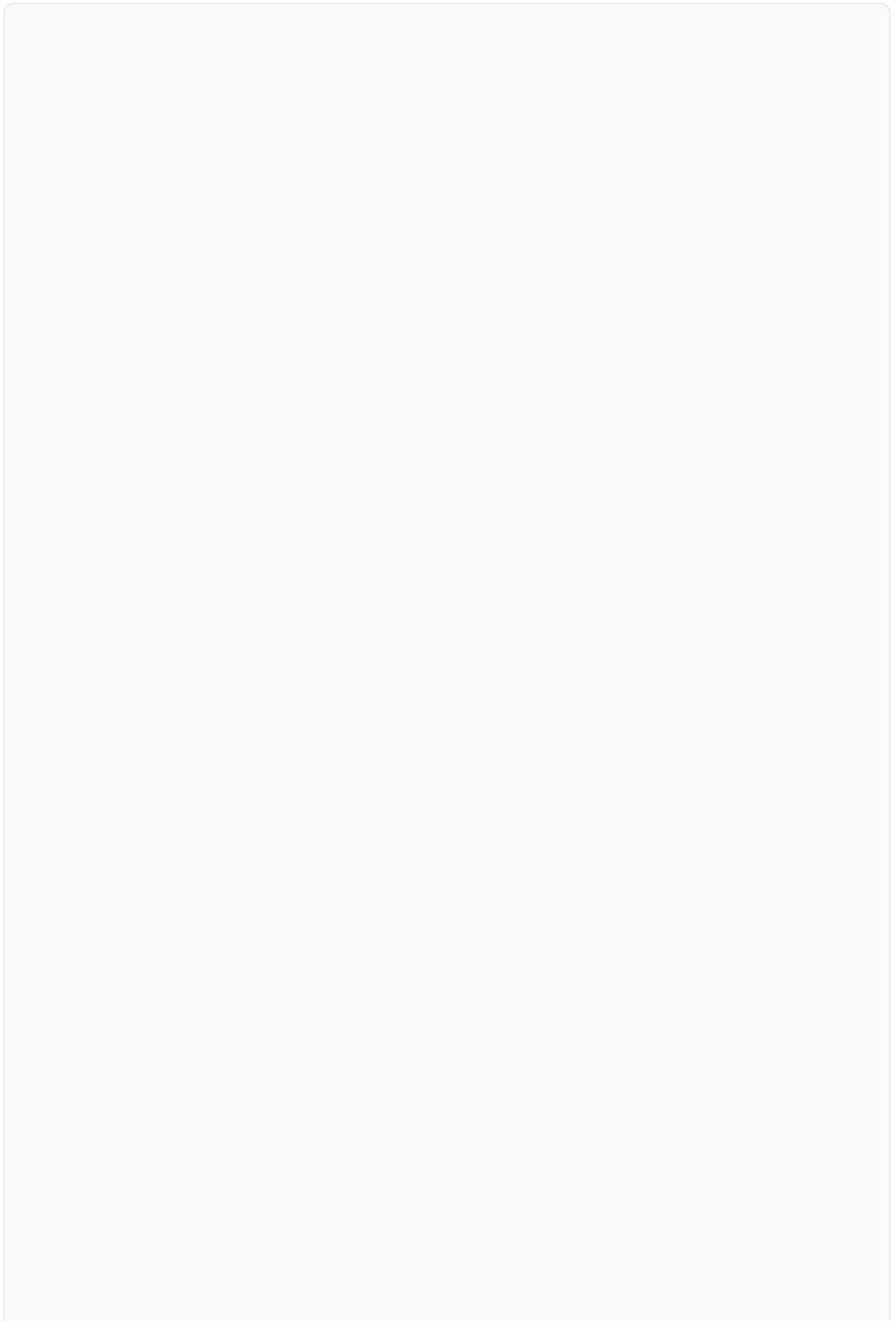


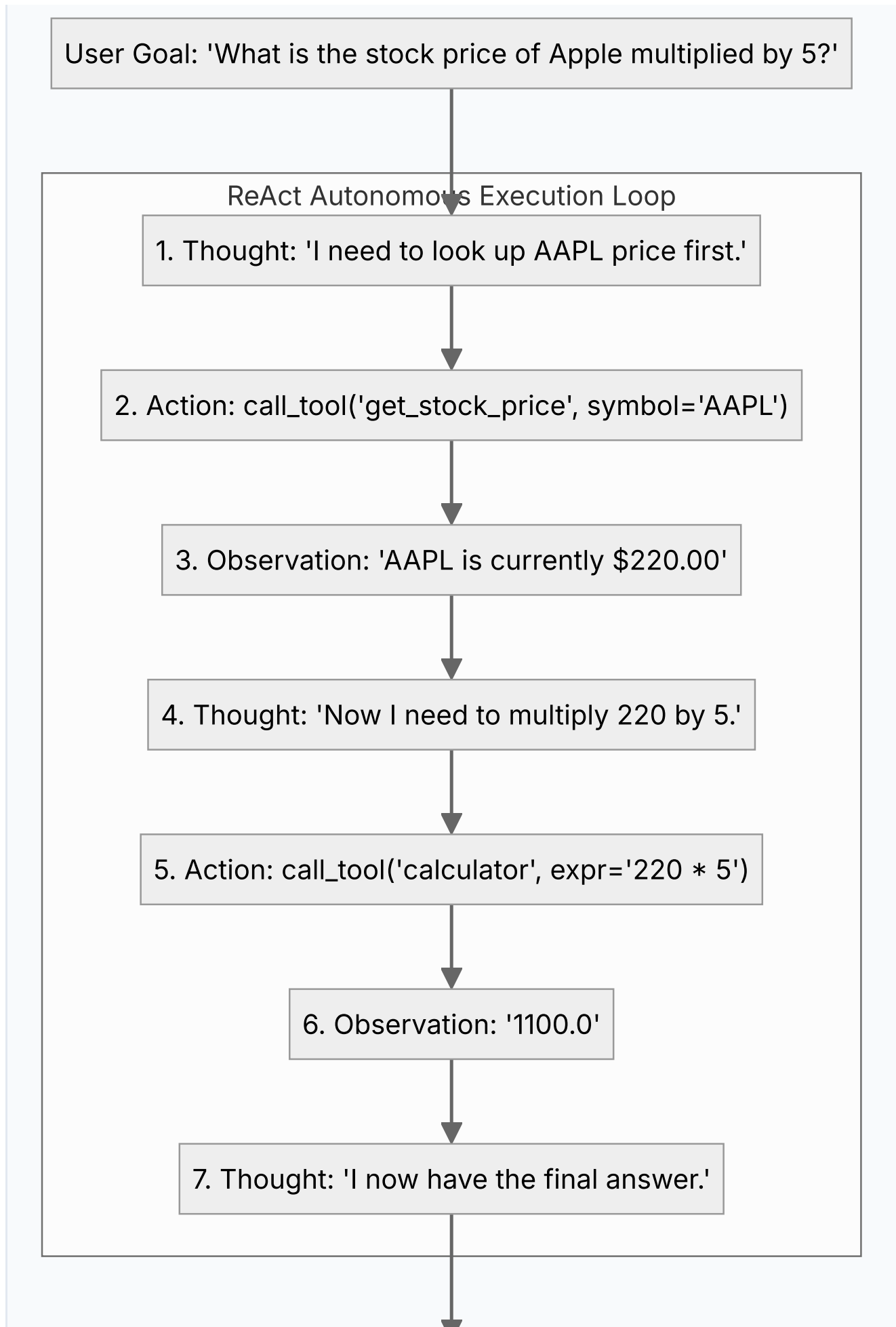
Agentic AI Chapter 4: Agent Cognitive Architectures: ReAct, Plan-and-Solve & Reflection

Core Learning Objective: Master the cognitive loops that drive autonomous AI agents. Build a working ReAct (Reasoning + Acting) loop from scratch, understand Plan-and-Solve architectures, and implement iterative self-correction with Reflexion.

1. The ReAct Pattern (Reasoning + Acting)

The **ReAct** architecture (Yao et al., 2022) combines internal reasoning traces with external tool actions in an iterative cycle:



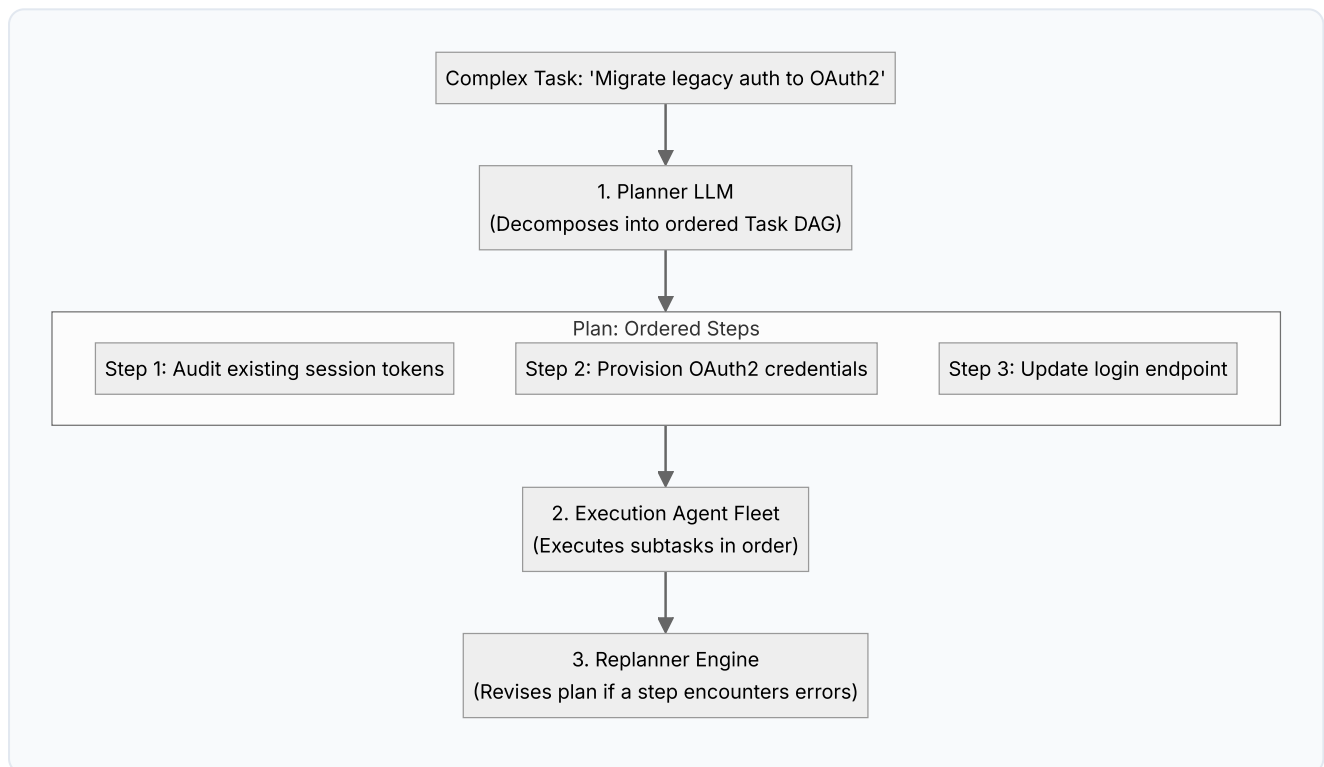


Final Response: '5 shares of Apple equal \$1,100.00'

Complete ReAct Loop Implementation in Python

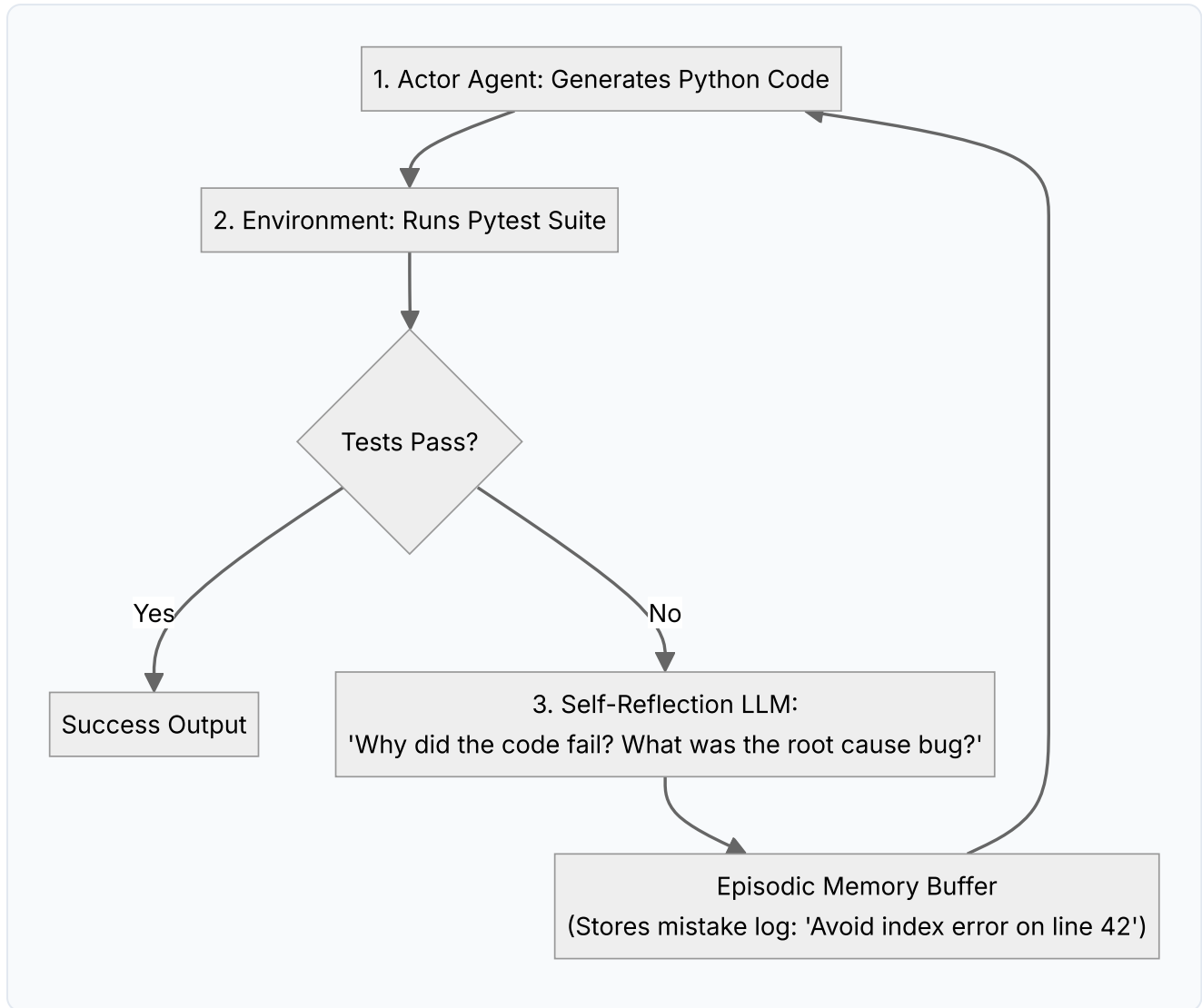
2. Plan-and-Solve Architecture

While ReAct decides each step greedily one by one (which can get lost in deep rabbit holes), **Plan-and-Solve** separates high-level planning from mechanical execution:



3. Reflexion: Iterative Self-Correction

Reflexion (Shinn et al., 2023) equips agents with an **evaluator and memory of failures**:



By reading its own previous mistakes in memory, the model refactors its code and avoids repeating bugs, increasing complex coding benchmark accuracy by > 30%.

4. Runnable Model: The Three Loops with Their Safety Limits

Every architecture above is a loop, and every production incident with agents is a loop that did not stop. These versions run offline with a scripted "model" so you can see the guards work.

Pattern selection. Use **ReAct** when the next step depends on what the last tool returned; **plan-and-solve** when the steps are knowable in advance (cheaper, easier to audit, and you can show the plan to a human before running it); **reflexion** when you have a reliable critic, such as unit tests or a schema validator. A critic that is itself a language model with no ground truth often reinforces the first answer rather than improving it.

Termination checklist for any agent loop

1. A hard **step budget** and a **token or cost budget**.
 2. **Repeated-call detection** (same tool, same arguments).
 3. A **wall-clock timeout** around every tool call.
 4. A defined behaviour when stopped: return partial work and say why, rather than raising a bare exception.
 5. Log the full trace; most agent bugs are diagnosed by reading it.
-

Further Reading

- [ReAct paper](#)
 - [Anthropic: building effective agents](#)
 - [Lilian Weng: LLM powered autonomous agents](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. ReAct in one sentence?

The model alternates reasoning steps and tool actions, using each observation to decide the next step.

▼ 2. Plan-and-solve versus ReAct?

Plan-and-solve writes a full plan first (fewer calls, reviewable); ReAct decides step by step (more adaptive).

▼ 3. Why do agents need step budgets?

To stop infinite loops and runaway cost when the model cannot make progress.

▼ 4. Why model an agent as a state machine?

Explicit states and transitions make behaviour testable, resumable and debuggable.